

Họ tên sinh viên: Hoàng Thị Minh Thu - 20252793M

**BÁO CÁO: Thực hành ảo hóa IoT - Lab work:
Week 2 - Day 1: Ảo hóa cho hệ thống IoT với
Docker, Mosquitto và Python**

Mục lục

I. Mục tiêu bài thực hành:	3
II. Kiến thức cần nắm:	3
III. Kiến trúc hệ thống thực hành:	3
IV. Chuẩn bị môi trường	4
1. Chuẩn bị môi trường:	4
2. Kiểm tra docker:	4
V. Phần 1: Chạy MQTT broker bằng Docker	4
1. Tải image Eclipse Mosquitto:	4
2. Tạo thư mục cấu hình Mosquitto	5
3. Chạy Mosquitto broker:	5
VI. Phần 2: Kiểm thử MQTT broker	6
1. Cách 1: Cài mqtt-client tools trên Ubuntu:	6
2. Cách 2: Chạy mqtt-client tools bên trong container:	6
3. Câu hỏi nhanh:	6
VII. Phần 3: Viết chương trình Python giả lập cảm biến IoT	7
1. Cấu trúc thư mục:	7
2. Nội dung requirement.txt:	7
3. Nội dung app.py:	8
VIII. Phần 4: Chạy chương trình Python trực tiếp trên host	9
1. Cài dependency:	9
IX. Phần 5: Container hóa sensor simulator	9
1. Viết Dockerfile:	9
2. Build Docker Image:	9
3. Tạo Docker Network:	10
4. Chạy sensor simulator container:	11
5. Subscribe dữ liệu từ host:	12
X. Phần 6: Chạy nhiều sensor simulator	12
XI. Phần 7: Quan sát và quản lý container	13
1. Container đang chạy:	13
2. Logs của container:	13
3. Kiểm tra network:	14
4. Dừng container:	14
XII. Phần 8: Tự xây dựng và Docker hóa ứng dụng IoT riêng	14
1. Cấu trúc thư mục và code:	14
2. Công dụng:	16
3. Kết quả chạy:	16
XIII. Câu hỏi báo cáo	18
XIV. Dọn dẹp môi trường	20
XV. Kết luận	21

I. Mục tiêu bài thực hành:

- + Hiểu vai trò của container trong triển khai ứng dụng IoT.
- + Chạy một MQTT broker bằng Docker container.
- + Sử dụng các công cụ dòng lệnh `mosquitto_pub` và `mosquitto_sub` để kiểm thử MQTT broker.
- + Viết chương trình Python giả lập cảm biến IoT.
- + Container hóa chương trình Python bằng Dockerfile.
- + Cho chương trình giả lập cảm biến publish dữ liệu dạng JSON lên MQTT broker.
- + Quan sát và kiểm tra dữ liệu MQTT được truyền giữa các thành phần

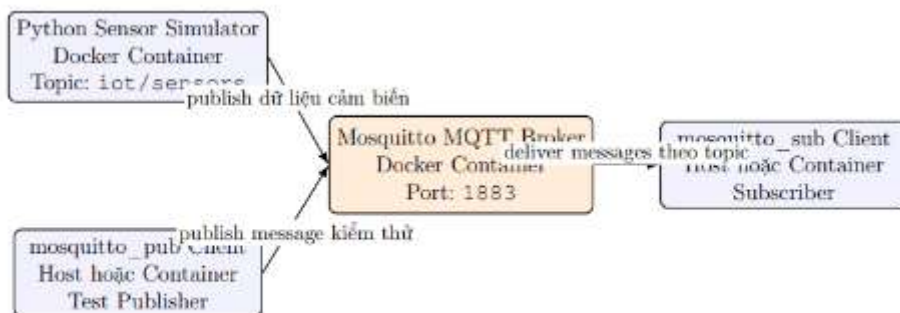
II. Kiến thức cần nắm:

Sinh viên cần nắm các kiến thức cơ bản sau:

- + Container: môi trường chạy ứng dụng được đóng gói cùng thư viện và dependency cần thiết.
- + Docker image: mẫu đóng gói ứng dụng, dùng để tạo container.
- + Docker container: instance đang chạy của một Docker image.
- + MQTT broker: thành phần trung gian nhận message từ publisher và chuyển tiếp đến subscriber.
- + MQTT publisher: client gửi message lên một topic.
- + MQTT subscriber: client đăng ký nhận message từ một topic.
- + Topic: kênh logic để phân loại message trong MQTT.

III. Kiến trúc hệ thống thực hành:

- Kiến trúc:



Hình 1: Kiến trúc MQTT tối thiểu trong bài lab Buổi 1

- Giải thích thành phần:

- + mqtt-broker: container chạy Eclipse Mosquitto, đóng vai trò MQTT broker trung gian.
- + sensor-simulator: chương trình Python giả lập cảm biến, đóng vai trò MQTT publisher.
- + mosquitto_pub: công cụ dòng lệnh dùng để publish thử một message lên MQTT broker.
- + mosquitto_sub: công cụ dòng lệnh dùng để subscribe một topic và nhận message từ MQTT broker

Trong MQTT, publisher và subscriber không trao đổi dữ liệu trực tiếp với nhau. Tất cả message đều đi qua MQTT broker.

IV. Chuẩn bị môi trường

1. Chuẩn bị môi trường:

- Môi trường sử dụng: Hệ điều hành Ubuntu, Docker, Docker Desktop, VS Code, Command prompt.

2. Kiểm tra docker:

- Kiểm tra version: trả về thành công

```
PS C:\Users\Tes\Desktop\LTAH-IoT\week2-lab1> docker --version
Docker version 29.4.2, build 055a478
PS C:\Users\Tes\Desktop\LTAH-IoT\week2-lab1> █
```

- Kiểm tra Docker daemon: chạy thành công

```
PS C:\Users\Tes\Desktop\LTAH-IoT\week2-lab1> docker run hello-world
Status: Downloaded newer image for hello-world:latest
```

Hello from Docker!

This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:

1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
(amd64)
3. The Docker daemon created a new container from that image which runs the executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it to your terminal.

To try something more ambitious, you can run an Ubuntu container with:

```
$ docker run -it ubuntu bash
```

Share images, automate workflows, and more with a free Docker ID:

<https://hub.docker.com/>

For more examples and ideas, visit:

<https://docs.docker.com/get-started/>

V. Phần 1: Chạy MQTT broker bằng Docker

1. Tải image Eclipse Mosquitto:

- Chạy lệnh tải Eclipse Mosquitto: tải thành công

```

PS C:\Users\Tes\Desktop\LTAA-IoT\week2-lab1> docker pull eclipse-mosquitto
Using default tag: latest
latest: Pulling from library/eclipse-mosquitto
f662e87835a0: Pull complete
08c22ad08fb4: Pull complete
8fe9b71886e2: Download complete
d025ccaf3faf: Download complete
Digest: sha256:a908c65cc8e67ec9d292ef27c2c0360dbaaee7eb1b935cdd194e67697f15dea1
Status: Downloaded newer image for eclipse-mosquitto:latest
docker.io/library/eclipse-mosquitto:latest

What's next:
View a summary of image vulnerabilities and recommendations → docker scout quickview eclipse-mosquitto
PS C:\Users\Tes\Desktop\LTAA-IoT\week2-lab1>

```

- Kiểm tra images đã được tải về: eclipse-mosquitto đã được tải về, hiển thị trong images.

```

PS C:\Users\Tes\Desktop\LTAA-IoT\week2-lab1> docker images

```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
adminer:latest	cc64d253db4f	170MB	46.7MB	U
alpine:latest	5b10f432ef3d	13.1MB	3.95MB	
confluentinc/cp-kafka:7.4.4	c0224a1adf7a	1.28GB	444MB	U
eclipse-mosquitto:latest	a908c65cc8e6	35.9MB	10.7MB	

2. Tạo thư mục cấu hình Mosquitto

- Cấu hình thư mục Mosquitto:

```

WEEK2-LAB1
> iot-sensor-simulator
  > mosquitto\config
    > mosquitto.conf

```

- Nội dung file mosquitto.conf:

```

mosquitto.conf X
mosquitto > config > mosquitto.conf
1 listener 1883
2 allow_anonymous true

```

3. Chạy Mosquitto broker:

- Chạy container Mosquitto - đặt tên mos-mqtt-broker:
- + Câu lệnh: `docker run -it --rm --name mos-mqtt-broker -p 1883:1883 -v "$PWD/mosquitto/config:/mosquitto/config:ro" eclipse-mosquitto:2`
- + Kết quả: port 1883 sẵn sàng, mosquitto version 2.1.2 running, mos-mqtt-broker đang chạy ở cổng 1883.

```
PS C:\Users\Tes\Desktop\LTAH-IoT\week2-lab1> docker run -it --name mos-mqtt-broker -p 1883:1883 -v "$PWD/mosquitto/config:/mosquitto/config:ro" eclipse-mosquitto:2
Unable to find image 'eclipse-mosquitto:2' locally
2: Pulling from library/eclipse-mosquitto
Digest: sha256:a908c65cc8e67ec9d292ef27c2c0360dbaaee7eb1b935cdd194e67697f15dea1
Status: Downloaded newer image for eclipse-mosquitto:2
1780921292: Info: running mosquitto as user: mosquitto.
1780921292: mosquitto version 2.1.2 starting
1780921292: Config loaded from /mosquitto/config/mosquitto.conf.
1780921292: Bridge support available.
1780921292: Persistence support available.
1780921292: TLS support available.
1780921292: TLS-PSK support available.
1780921292: Websockets support available.
1780921292: Opening ipv4 listen socket on port 1883.
1780921292: Opening ipv6 listen socket on port 1883.
1780921292: mosquitto version 2.1.2 running
```

□ ● mos-mqtt-broker 0d3877700fc4 eclipse-mo 1883:1883 0.0 📦 🗑

VI. Phần 2: Kiểm thử MQTT broker

Có 2 cách kiểm thử mqtt broker:

- Cách 1: cài mosquitto-client trên host
- Cách 2: chạy mosquitto_pub và mosquitto_sub trực tiếp bên trong broker container.

1. Cách 1: Cài mqtt-client tools trên Ubuntu:

- Cài đặt thành công mosquitto-client bằng ubuntu: truy cập vào Ubuntu bằng wsl -d Ubuntu -> sau ý thực hiện lệnh các lệnh sudo apt update -> sudo apt install mosquitto-client -y

```
Processing triggers for libc-bin (2.43-2ubuntu2) ...
thuiy05@Thuiy08:/mnt/c/WINDOWS/system32$ mosquitto_pub --version
mosquitto_pub version 2.0.22 running on libmosquitto 2.0.22.
thuiy05@Thuiy08:/mnt/c/WINDOWS/system32$
```

- Kiểm tra publish và subscribe (trước khi chạy phải truy cập vào ubuntu bằng câu lệnh wsl -d Ubuntu):

```
PS C:\Users\Tes> wsl -d Ubuntu
thuiy05@Thuiy08:/mnt/c/Users/Tes$ mosquitto_pub -h localhost -p 1883 -t 'iot/temperature' -m '28.5'
thuiy05@Thuiy08:/mnt/c/Users/Tes$
```

```
PS C:\Users\Tes> wsl -d Ubuntu
thuiy05@Thuiy08:/mnt/c/Users/Tes$ mosquitto_sub -h localhost -p 1883 -t 'iot/temperature'
28.5
```

2. Cách 2: Chạy mqtt-client tools bên trong container:

- Chạy các lệnh sau: docker exec -it mos-mqtt-broker -> mosquitto_sub -h localhost -p 1883 -t 'iot/temperature' và mosquitto_pub -h localhost -p 1883 -t 'iot/temperature' -m '28.5'

```
PS C:\Users\Tes\Desktop\LTAH-IoT\week2-lab1> docker exec -it mos-mqtt-broker mosquitto_pub -h localhost -p 1883 -t 'iot/temperature' -m '28.5'
```

What's next:
Try Docker Debug for seamless, persistent debugging tools in any container or image → [docker debug mos-mqtt-broker](https://docs.docker.com/go/debug-cli/)
Learn more at <https://docs.docker.com/go/debug-cli/>

```
PS C:\Users\Tes\Desktop\LTAH-IoT\week2-lab1> docker exec -it mos-mqtt-broker mosquitto_sub -h localhost -p 1883 -t 'iot/temperature'
28.5
```

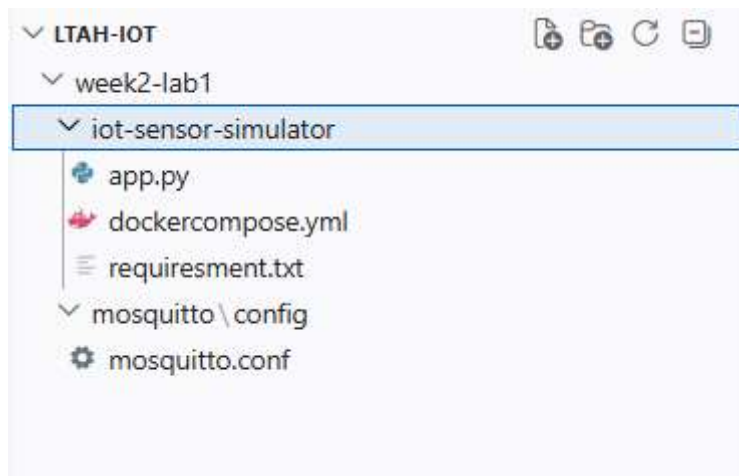
3. Câu hỏi nhanh:

- **Mosquitto_pub có vai trò gì?** Tạo message gửi đến mos_mqtt_broker cho topic 'iot/temperature'.
- **Mosquitto_sub có vai trò gì?** Đăng ký với mos_mqtt_broker để nhận messages từ topic 'iot/temperature'.

- **Mqtt-broker có trực tiếp tạo dữ liệu cảm biến không?** Không, mqtt-broker chỉ đóng vai trò trung gian giữa publisher và subscriber của các topic.
- **Topic `iot/temperature` có tác dụng gì?** Topic là địa chỉ định danh để broker phân loại và định tuyến message. Publisher gửi lên topic nào, subscriber đăng ký topic đó mới nhận được. Topic là 1 kênh truyền dữ liệu có tên cụ thể.
- **Vì sao cần ánh xạ cổng bằng tham số `-p 1883:1883`?** Container chạy bên trong Docker bị cô lập, bên ngoài không truy cập được. Ánh xạ cổng để host (Windows) và các thiết bị ngoài có thể kết nối vào broker qua cổng 1883. Dùng cùng số cổng để tránh nhầm lẫn.

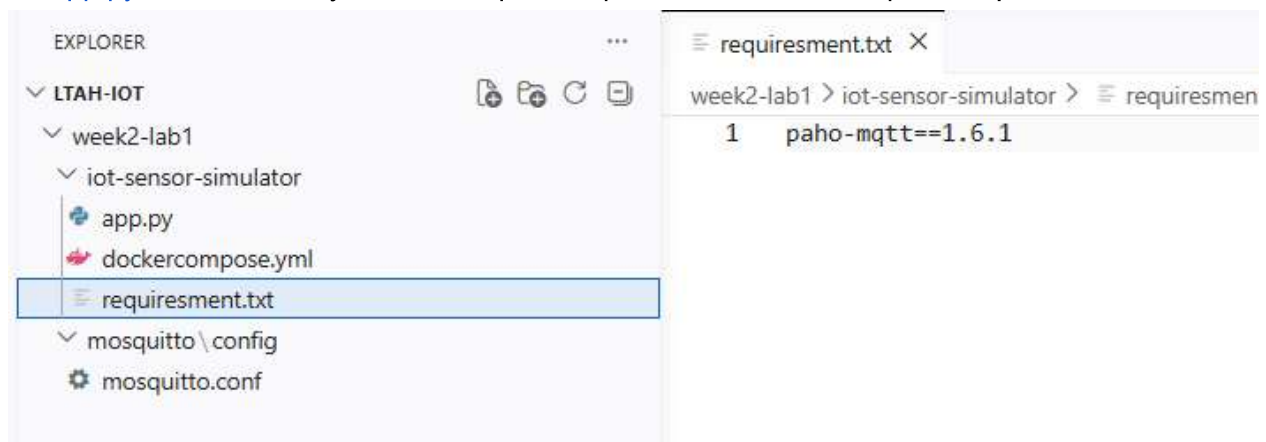
VII. Phần 3: Viết chương trình Python giả lập cảm biến IoT

1. Cấu trúc thư mục:



2. Nội dung requirement.txt:

Để [app.py](#) có thể nói chuyện với mosquitto mqtt thì cần tới thư viện paho-mqtt



3. Nội dung `app.py`:

```
1 import json
2 import os
3 import random
4 import time
5 from datetime import datetime, timezone
6 #datetime, timezone nằm trong module datetime của python
7
8 import paho.mqtt.client as mqtt
9
10 #os.getenv là hàm trong module os, dùng để đọc giá trị của 1 biến môi trường trong hệ điều hành
11 #cú pháp: os.getenv('TÊN BIẾN', 'giá trị mặc định');
12 #biến môi trường là gì? Là các giá trị cấu hình được lưu bên ngoài code, ở cấp hệ điều hành
13 #vd: trước khi chạy chương trình, bạn có thể đặt
14 #trên terminal (Linux/Mac): export MQTT_BROKER = 192.168.1.100; Windows: $env:MQTT_BROKER = '192.169.1.100'
15 #khi đó os.getenv('MQTT_BROKER', 'localhost') sẽ trả về 192.168.1.100 thay vì localhost
16 MQTT_BROKER = os.getenv('MQTT_BROKER', 'localhost') #địa chỉ của mqtt_broker: mặc định là localhost
17 MQTT_PORT = int(os.getenv('MQTT_PORT', '1883')) #os.getenv trả về string, sử dụng int(os.getenv...) giúp giá trị trả về dưới dạng int
18 MQTT_TOPIC = os.getenv('MQTT_TOPIC', 'iot/sensors')
19 DEVICE_ID = os.getenv('DEVICE_ID', 'sensor-01')
20 PUBLISH_INTERVAL = float(os.getenv('PUBLISH_INTERVAL', '2'))
21
22 #Trong python, sau khi khai báo hàm với dấu : ở cuối,
23 #tất cả các dòng thuộc thân hàm phải được thụt vào trong (thường là 4 dấu cách)
24 def create_sensor_data():
25     #tạo giá trị mô phỏng cho sensor
26     temperature = round(random.uniform(25.0,35.0),2)
27     humidity = round(random.uniform(50.0,90.0),2)
28     #uniform chỉ nhận 2 tham số còn 2 là của round
29
30     #trong dictionary (từ điển) phải dùng dấu , chứ không phải dấu ;
31     #dictionary là 1 kiểu dữ liệu trong Python để lưu trữ dữ liệu theo dạng cặp key: value
32     data = {
33         'device_id': DEVICE_ID,
34         'temperature': temperature,
35         'humidity': humidity,
36         'timestamp': datetime.now(timezone.utc).isoformat()
37     }
38
39     return data
40
41 def main(): #định nghĩa hàm main, là hàm điều chỉnh toàn bộ luồng chạy của chương trình (kết nối -> tạo dữ liệu -> gửi đi)
42     #tạo 1 mqtt client
43     client = mqtt.Client(client_id=DEVICE_ID)
44     #in thông báo ra màn hình nhờ f-string - cho phép chèn giá trị biến vào trong {...}
45     print(f'Connecting to mqtt broker at {MQTT_BROKER}:{MQTT_PORT}...')
46     #thực hiện kết nối tới broker, mở kết nối 60s, nếu trong 60s không có dữ liệu trao đổi, client sẽ ping để báo 'tôi vẫn sống',
47     #nếu broker không nhận được -> coi như client mất kết nối
48     client.connect(MQTT_BROKER, MQTT_PORT, keepalive=60)
49
50     print(f"Publishing data to topic: {MQTT_TOPIC}")
51
52     while True:
53         data = create_sensor_data(1)
54         payload = json.dumps(data)
55
56         result = client.publish(MQTT_TOPIC, payload)
57         if result.rc == mqtt.MQTT_ERR_SUCCESS:
58             print(f'Publisher: {payload}')
59         else:
60             print(f'Failed to publish message. Error code: {result.rc}')
61
62         time.sleep(PUBLISH_INTERVAL)
63
64 if __name__ == '__main__':
65     main()
66
```


VIII. Phần 4: Chạy chương trình Python trực tiếp trên host

1. Cài dependency:

```
PS D:\LTAH-IoT\week2-lab1\iot-sensor-simulator> pip install -r requirements.txt
Building wheels for collected packages: paho-mqtt
  Building wheel for paho-mqtt (pyproject.toml) ... done
  Created wheel for paho-mqtt: filename=paho_mqtt-1.6.1-py3-none-any.whl size=65581 sha256=0ceac806af6e05533a0a674d17c50d0443ac5f3e05e51f436f8bb8affddd6e3b
  Stored in directory: c:\users\tes\appdata\local\pip\cache\wheels\e2\cb\cb\00c9b1e003f03b991bf5fa83bfe4efcd8bb1a983caab502473
Successfully built paho-mqtt
Installing collected packages: paho-mqtt
Successfully installed paho-mqtt-1.6.1

[notice] A new release of pip is available: 26.1.1 -> 26.1.2
[notice] To update, run: C:\Users\Tes\AppData\Local\Python\pythoncore-3.14-64\python.exe -m pip install --upgrade pip
PS D:\LTAH-IoT\week2-lab1\iot-sensor-simulator>
```

2. Chạy sensor simulator:

```
PS D:\LTAH-IoT\week2-lab1\iot-sensor-simulator> python app.py
Connecting to mqtt broker at localhost:1883...
Publishing data to topic: iot/sensors
Publisher: {"device_id": "sensor-01", "temperature": 33.81, "humidity": 66.15, "timestamp": "2026-06-15T04:30:57.253518+00:00"}
Publisher: {"device_id": "sensor-01", "temperature": 31.94, "humidity": 55.01, "timestamp": "2026-06-15T04:30:59.262200+00:00"}
Publisher: {"device_id": "sensor-01", "temperature": 29.41, "humidity": 62.06, "timestamp": "2026-06-15T04:31:01.263127+00:00"}
Publisher: {"device_id": "sensor-01", "temperature": 31.11, "humidity": 50.21, "timestamp": "2026-06-15T04:31:03.264173+00:00"}
Publisher: {"device_id": "sensor-01", "temperature": 34.55, "humidity": 67.4, "timestamp": "2026-06-15T04:31:05.265468+00:00"}

```

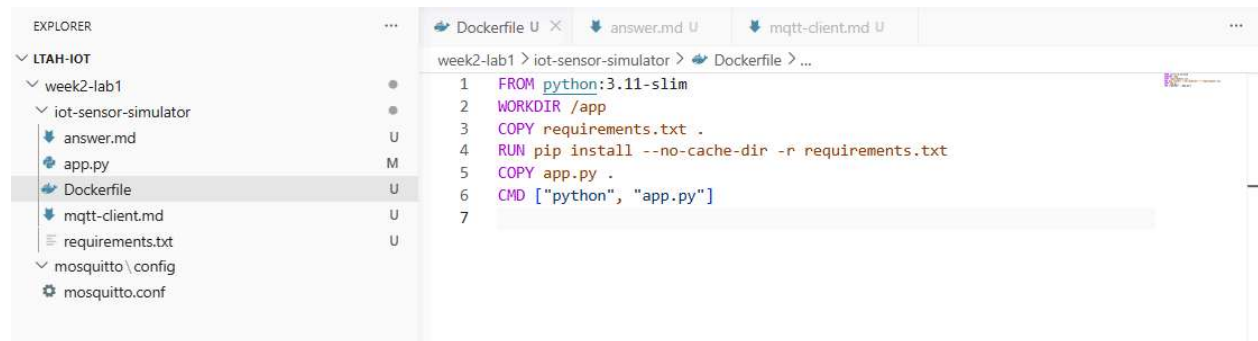
3. Subscribe để kiểm tra dữ liệu:

```
thuiy05@Thuiy08:~$ mosquitto_sub -h localhost -p 1883 -t 'iot/sensors'
{"device_id": "sensor-01", "temperature": 30.61, "humidity": 59.03, "timestamp": "2026-06-15T04:33:03.383189+00:00"}
{"device_id": "sensor-01", "temperature": 32.9, "humidity": 67.66, "timestamp": "2026-06-15T04:33:05.383945+00:00"}
{"device_id": "sensor-01", "temperature": 28.23, "humidity": 65.16, "timestamp": "2026-06-15T04:33:07.384676+00:00"}

```

IX. Phần 5: Container hóa sensor simulator

1. Viết Dockerfile:



2. Build Docker Image:

In iot-sensor-simulator run: `docker build -t iot-sensor-simulator:1.0 .`

```
PS D:\LTAH-IoT\week2-lab1\iot-sensor-simulator> docker build -t iot-sensor-simulator:1.0 .
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:827f49578c11a6255a052268401f7e03a2fae6aaff5e09b884e9a6e5e2ee07bc
=> => exporting config sha256:b5052a34e288576716de446ee4ebbf4308d7c409c7ebc75246b74e7cd3434f52
=> => exporting attestation manifest sha256:bff886e9f547df43905a5ea3651ffe2b71abbc40f0ca2a43b33f5ada1cc84c04
=> => exporting manifest list sha256:e090d3371f5ec49512b100e0b48db57e3408f853da57d38a9d2c0549f284350e
=> => naming to docker.io/library/iot-sensor-simulator:1.0
=> => unpacking to docker.io/library/iot-sensor-simulator:1.0

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/vhc8gylhdgm69s1auds69c6j
PS D:\LTAH-IoT\week2-lab1\iot-sensor-simulator>
```

Kiểm tra bằng lệnh `docker images` -> iot-sensor-simulator images đã được tạo

```
PS D:\LTAH-IoT\week2-lab1\iot-sensor-simulator> docker images
```

Repository	Tag	Image ID	Size	Virtual Size	Platform
golang	1.24	d2d2bc1c84f7	1.32GB	335MB	linux/amd64
golang	1.25	4224e352b083	1.26GB	316MB	linux/amd64
hello-world	latest	0e760fdfbc48	25.9kB	9.49kB	linux/amd64
iot-sensor-simulator	1.0	e090d3371f5e	207MB	50.4MB	linux/amd64

3. Tạo Docker Network:

Tạo mạng docker:

```
PS D:\LTAH-IoT\week2-lab1\iot-sensor-simulator> docker network create iot-net
2e669b01ed2574d7aaa6839ed2f324cc5e6860523f10ff9ded20bda4690b2def
PS D:\LTAH-IoT\week2-lab1\iot-sensor-simulator>
```

Check -> đã thấy mạng iot-net:

```
PS D:\LTAH-IoT\week2-lab1\iot-sensor-simulator> docker network ls
```

NETWORK ID	NAME	DRIVER	SCOPE
edea8c63a97a	bridge	bridge	local
944ec0239520	host	host	local
2e669b01ed25	iot-net	bridge	local
c7dc63f62faa	nio_default	bridge	local
eaad358e3a6b	none	null	local

```
PS D:\LTAH-IoT\week2-lab1\iot-sensor-simulator>
```

Kiểm tra thông tin iot-net

```

eaad358e3a6b  none      null      local
PS D:\LTAH-IoT\week2-lab1\iot-sensor-simulator> docker network inspect iot-net
[
  {
    "Name": "iot-net",
    "Id": "2e669b01ed2574d7aaa6839ed2f324cc5e6860523f10ff9ded20bda4690b2def",
    "Created": "2026-06-15T04:58:20.302453504Z",
    "Scope": "local",
    "Driver": "bridge",
    "EnableIPv4": true,
    "EnableIPv6": false,
    "IPAM": {
      "Driver": "default",
      "Options": {},
      "Config": [
        {
          "Subnet": "172.19.0.0/16",
          "Gateway": "172.19.0.1"
        }
      ]
    },
    "Internal": false,
    "Attachable": false,
    "Ingress": false,
    "ConfigFrom": {
      "Network": ""
    },
    "ConfigOnly": false,
    "Options": {
      "com.docker.network.enable_ipv4": "true",
      "com.docker.network.enable_ipv6": "false"
    },
    "Labels": {},
    "Containers": {},
    "Status": {
      "IPAM": {
        "Subnets": {
          "172.19.0.0/16": {
            "IPsInUse": 3,
            "DynamicIPsAvailable": 65533
          }
        }
      }
    }
  }
]

```

Chạy mqtt-broker trong iot-net:

```

PS D:\LTAH-IoT\week2-lab1> docker run -it --rm --name mos-mqtt-broker --network iot-net -p 1883:1883 -v "$PWD/mosquitto/config:/mosquitto/config:ro" eclipse-mosquitto
:2
1781499668: Info: running mosquitto as user: mosquitto.
1781499668: mosquitto version 2.1.2 starting
1781499668: Config loaded from /mosquitto/config/mosquitto.conf.
1781499668: Bridge support available.
1781499668: Persistence support available.
1781499668: TLS support available.
1781499668: TLS-PSK support available.
1781499668: Websockets support available.
1781499668: Opening ipv4 listen socket on port 1883.
1781499668: Opening ipv6 listen socket on port 1883.
1781499668: mosquitto version 2.1.2 running

```

4. Chạy sensor simulator container:

Chạy lệnh: `docker run -it --rm --name sensor-01 --network iot-net -e MQTT_BROKER=mos-mqtt-broker -e MQTT_PORT=1883 -e MQTT_TOPIC=iot/sensors -e DEVICE_ID=sensor-01 -e PUBLISH_INTERVAL=2 iot-sensor-simulator:1.0`


```
PS D:\LTAH-IoT\week2-lab1\iot-sensor-simulator> docker run -it --rm --name sensor-01 --network iot-net -e MQTT_BROKER=mos-mqtt-broker -e MQTT_PORT=1883 -e MQTT_TOPIC=i
ot/sensors -e DEVICE_ID=sensor-01 -e PUBLISH_INTERVAL=2 iot-sensor-simulator:1.0
Connecting to mqtt broker at mos-mqtt-broker:1883...
Publishing data to topic: iot/sensors
Publisher: {"device_id": "sensor-01", "temperature": 25.53, "humidity": 57.01, "timestamp": "2026-06-15T05:04:59.050667+00:00"}
Publisher: {"device_id": "sensor-01", "temperature": 27.43, "humidity": 55.6, "timestamp": "2026-06-15T05:05:01.051224+00:00"}
Publisher: {"device_id": "sensor-01", "temperature": 25.22, "humidity": 79.83, "timestamp": "2026-06-15T05:05:03.051922+00:00"}
█
```

5. Subscribe dữ liệu từ host:

```
Use 'mosquitto_sub --help' to see usage.
thuiy05@Thuiy08:/mnt/c/WINDOWS/system32$ mosquitto_sub -h localhost -p 1883 -t 'iot/sensors'
{"device_id": "sensor-01", "temperature": 26.57, "humidity": 73.44, "timestamp": "2026-06-15T05:08:49.182963+00:00"}
{"device_id": "sensor-01", "temperature": 32.92, "humidity": 69.1, "timestamp": "2026-06-15T05:08:51.183573+00:00"}
{"device_id": "sensor-01", "temperature": 33.0, "humidity": 89.54, "timestamp": "2026-06-15T05:08:53.184137+00:00"}
{"device_id": "sensor-01", "temperature": 32.4, "humidity": 73.43, "timestamp": "2026-06-15T05:08:55.185171+00:00"}
{"device_id": "sensor-01", "temperature": 29.29, "humidity": 85.83, "timestamp": "2026-06-15T05:08:57.186009+00:00"}
{"device_id": "sensor-01", "temperature": 31.77, "humidity": 83.06, "timestamp": "2026-06-15T05:08:59.187238+00:00"}
{"device_id": "sensor-01", "temperature": 26.88, "humidity": 89.51, "timestamp": "2026-06-15T05:09:01.188465+00:00"}
█
```

X. Phần 6: Chạy nhiều sensor simulator

Chạy publisher: 01-02-03

```
PS D:\LTAH-IoT\week2-lab1\iot-sensor-simulator> docker run -it --rm --name sensor-01 --network iot-net -e MQTT_BROKER=mos-mqtt-broker -e MQTT_PORT=1883 -e MQTT_TOPIC=i
ot/sensors -e DEVICE_ID=sensor-01 -e PUBLISH_INTERVAL=2 iot-sensor-simulator:1.0
Connecting to mqtt broker at mos-mqtt-broker:1883...
Publishing data to topic: iot/sensors
Publisher: {"device_id": "sensor-01", "temperature": 27.22, "humidity": 56.67, "timestamp": "2026-06-15T05:09:27.606740+00:00"}
Publisher: {"device_id": "sensor-01", "temperature": 28.43, "humidity": 87.0, "timestamp": "2026-06-15T05:09:29.614683+00:00"}
Publisher: {"device_id": "sensor-01", "temperature": 34.06, "humidity": 58.08, "timestamp": "2026-06-15T05:09:31.615761+00:00"}
Publisher: {"device_id": "sensor-01", "temperature": 31.16, "humidity": 63.46, "timestamp": "2026-06-15T05:09:33.616633+00:00"}
Publisher: {"device_id": "sensor-01", "temperature": 28.63, "humidity": 69.77, "timestamp": "2026-06-15T05:09:35.617305+00:00"}
Publisher: {"device_id": "sensor-01", "temperature": 27.66, "humidity": 56.12, "timestamp": "2026-06-15T05:09:37.618009+00:00"}
Publisher: {"device_id": "sensor-01", "temperature": 28.92, "humidity": 67.43, "timestamp": "2026-06-15T05:09:39.618843+00:00"}
Publisher: {"device_id": "sensor-01", "temperature": 33.11, "humidity": 53.22, "timestamp": "2026-06-15T05:09:41.617060+00:00"}
Publisher: {"device_id": "sensor-01", "temperature": 29.26, "humidity": 61.92, "timestamp": "2026-06-15T05:09:43.617726+00:00"}
Publisher: {"device_id": "sensor-01", "temperature": 29.49, "humidity": 75.78, "timestamp": "2026-06-15T05:09:45.618352+00:00"}
Publisher: {"device_id": "sensor-01", "temperature": 32.5, "humidity": 86.55, "timestamp": "2026-06-15T05:09:47.618887+00:00"}
Publisher: {"device_id": "sensor-01", "temperature": 25.66, "humidity": 60.51, "timestamp": "2026-06-15T05:09:49.620355+00:00"}
Publisher: {"device_id": "sensor-01", "temperature": 28.75, "humidity": 70.25, "timestamp": "2026-06-15T05:09:51.621010+00:00"}
█
```

```
PS D:\LTAH-IoT\week2-lab1\iot-sensor-simulator> docker run -it --rm --name sensor-02 --network iot-net -e MQTT_BROKER=mos-mqtt-broker -e MQTT_PORT=1883 -e MQTT_TOPIC=i
ot/sensors -e DEVICE_ID=sensor-01 -e PUBLISH_INTERVAL=2 iot-sensor-simulator:1.0
Publisher: {"device_id": "sensor-01", "temperature": 26.16, "humidity": 56.18, "timestamp": "2026-06-15T05:10:30.073314+00:00"}
Publisher: {"device_id": "sensor-01", "temperature": 27.49, "humidity": 69.92, "timestamp": "2026-06-15T05:10:32.073924+00:00"}
Publisher: {"device_id": "sensor-01", "temperature": 30.5, "humidity": 57.93, "timestamp": "2026-06-15T05:10:34.074569+00:00"}
█
```

```
PS D:\LTAH-IoT\week2-lab1\iot-sensor-simulator> docker run -it --rm --name sensor-03 --network iot-net -e MQTT_BROKER=mos-mqtt-broker -e MQTT_PORT=1883 -e MQTT_TOPIC=i
ot/sensors -e DEVICE_ID=sensor-01 -e PUBLISH_INTERVAL=2 iot-sensor-simulator:1.0
Publishing data to topic: iot/sensors
Publisher: {"device_id": "sensor-01", "temperature": 31.5, "humidity": 88.05, "timestamp": "2026-06-15T05:10:32.622220+00:00"}
Publisher: {"device_id": "sensor-01", "temperature": 30.43, "humidity": 51.24, "timestamp": "2026-06-15T05:10:34.622919+00:00"}
Publisher: {"device_id": "sensor-01", "temperature": 34.99, "humidity": 64.36, "timestamp": "2026-06-15T05:10:36.623669+00:00"}
Publisher: {"device_id": "sensor-01", "temperature": 29.49, "humidity": 56.06, "timestamp": "2026-06-15T05:10:38.624455+00:00"}
Publisher: {"device_id": "sensor-01", "temperature": 28.39, "humidity": 60.82, "timestamp": "2026-06-15T05:10:40.623349+00:00"}
█
```

Subscriber nhận đủ dữ liệu của sensor 01, 02, 03:

```
{
  "device_id": "sensor-01",
  "temperature": 26.31,
  "humidity": 65.41,
  "timestamp": "2026-06-15T05:13:02.687644+00:00"
}
{
  "device_id": "sensor-01",
  "temperature": 33.96,
  "humidity": 73.29,
  "timestamp": "2026-06-15T05:13:04.688399+00:00"
}
{
  "device_id": "sensor-01",
  "temperature": 26.16,
  "humidity": 61.55,
  "timestamp": "2026-06-15T05:13:06.689093+00:00"
}
{
  "device_id": "sensor-01",
  "temperature": 26.85,
  "humidity": 78.65,
  "timestamp": "2026-06-15T05:13:08.689674+00:00"
}
{
  "device_id": "sensor-01",
  "temperature": 29.01,
  "humidity": 60.44,
  "timestamp": "2026-06-15T05:13:10.684782+00:00"
}
{
  "device_id": "sensor-01",
  "temperature": 28.58,
  "humidity": 57.61,
  "timestamp": "2026-06-15T05:13:12.685378+00:00"
}
{
  "device_id": "sensor-01",
  "temperature": 27.61,
  "humidity": 88.39,
  "timestamp": "2026-06-15T05:13:14.685998+00:00"
}
{
  "device_id": "sensor-01",
  "temperature": 29.3,
  "humidity": 67.44,
  "timestamp": "2026-06-15T05:13:16.686516+00:00"
}
{
  "device_id": "sensor-01",
  "temperature": 29.54,
  "humidity": 73.02,
  "timestamp": "2026-06-15T05:13:18.686979+00:00"
}
{
  "device_id": "sensor-01",
  "temperature": 28.23,
  "humidity": 72.34,
  "timestamp": "2026-06-15T05:13:20.687607+00:00"
}
{
  "device_id": "sensor-01",
  "temperature": 32.78,
  "humidity": 86.72,
  "timestamp": "2026-06-15T05:13:22.688236+00:00"
}
{
  "device_id": "sensor-01",
  "temperature": 32.39,
  "humidity": 78.65,
  "timestamp": "2026-06-15T05:13:24.688874+00:00"
}
{
  "device_id": "sensor-01",
  "temperature": 28.97,
  "humidity": 81.23,
  "timestamp": "2026-06-15T05:13:26.689342+00:00"
}
{
  "device_id": "sensor-01",
  "temperature": 25.07,
  "humidity": 66.01,
  "timestamp": "2026-06-15T05:13:28.689829+00:00"
}
{
  "device_id": "sensor-01",
  "temperature": 28.86,
  "humidity": 72.66,
  "timestamp": "2026-06-15T05:13:30.690382+00:00"
}
{
  "device_id": "sensor-03",
  "temperature": 33.05,
  "humidity": 86.99,
  "timestamp": "2026-06-15T05:13:37.910482+00:00"
}
{
  "device_id": "sensor-03",
  "temperature": 25.57,
  "humidity": 68.3,
  "timestamp": "2026-06-15T05:13:39.911988+00:00"
}
{
  "device_id": "sensor-03",
  "temperature": 25.03,
  "humidity": 62.25,
  "timestamp": "2026-06-15T05:13:41.910951+00:00"
}
{
  "device_id": "sensor-03",
  "temperature": 34.39,
  "humidity": 80.1,
  "timestamp": "2026-06-15T05:13:43.912220+00:00"
}
{
  "device_id": "sensor-03",
  "temperature": 34.27,
  "humidity": 56.69,
  "timestamp": "2026-06-15T05:13:45.913348+00:00"
}
{
  "device_id": "sensor-02",
  "temperature": 27.08,
  "humidity": 62.84,
  "timestamp": "2026-06-15T05:13:47.653200+00:00"
}
{
  "device_id": "sensor-03",
  "temperature": 32.09,
  "humidity": 54.91,
  "timestamp": "2026-06-15T05:13:49.913842+00:00"
}
{
  "device_id": "sensor-02",
  "temperature": 26.78,
  "humidity": 55.47,
  "timestamp": "2026-06-15T05:13:49.656986+00:00"
}
{
  "device_id": "sensor-03",
  "temperature": 26.41,
  "humidity": 77.83,
  "timestamp": "2026-06-15T05:13:49.914509+00:00"
}
{
  "device_id": "sensor-02",
  "temperature": 34.62,
  "humidity": 70.94,
  "timestamp": "2026-06-15T05:13:51.658032+00:00"
}
{
  "device_id": "sensor-03",
  "temperature": 29.56,
  "humidity": 56.14,
  "timestamp": "2026-06-15T05:13:51.915292+00:00"
}
{
  "device_id": "sensor-02",
  "temperature": 33.34,
  "humidity": 74.65,
  "timestamp": "2026-06-15T05:13:53.658885+00:00"
}
{
  "device_id": "sensor-03",
  "temperature": 28.28,
  "humidity": 52.84,
  "timestamp": "2026-06-15T05:13:53.916070+00:00"
}
```

XI. Phần 7: Quan sát và quản lý container

1. Container đang chạy:

CONTAINER ID	IMAGE	COMMAND NAMES	CREATED	STATUS	PORTS
ca26568ab833	iot-sensor-simulator:1.0	"python app.py" sensor-02	7 seconds ago	Up 6 seconds	
e9d08445ac78	iot-sensor-simulator:1.0	"python app.py" sensor-03	About a minute ago	Up About a minute	
968f51119d35	iot-sensor-simulator:1.0	"python app.py" sensor-01	3 minutes ago	Up 2 minutes	
9a5c67120897	eclipse-mosquitto:2	"/docker-entrypoint..."	13 minutes ago	Up 13 minutes	0.0.0.0:1883->1883/tcp, [::]:1883->1883/tcp

2. Logs của container:

```
PS D:\LTAH-IoT> docker logs sensor-01
Connecting to mqtt broker at mos-mqtt-broker:1883...
Publishing data to topic: iot/sensors
Publisher: {"device_id": "sensor-01", "temperature": 29.02, "humidity": 81.1, "timestamp": "2026-06-15T05:11:46.267034+00:00"}
Publisher: {"device_id": "sensor-01", "temperature": 31.97, "humidity": 76.76, "timestamp": "2026-06-15T05:11:48.270709+00:00"}
Publisher: {"device_id": "sensor-01", "temperature": 25.46, "humidity": 50.94, "timestamp": "2026-06-15T05:11:50.271499+00:00"}
Publisher: {"device_id": "sensor-01", "temperature": 33.11, "humidity": 51.77, "timestamp": "2026-06-15T05:11:52.272283+00:00"}
Publisher: {"device_id": "sensor-01", "temperature": 33.2, "humidity": 61.79, "timestamp": "2026-06-15T05:11:54.277442+00:00"}
Publisher: {"device_id": "sensor-01", "temperature": 29.49, "humidity": 89.6, "timestamp": "2026-06-15T05:11:56.278488+00:00"}
```

3. Kiểm tra network:

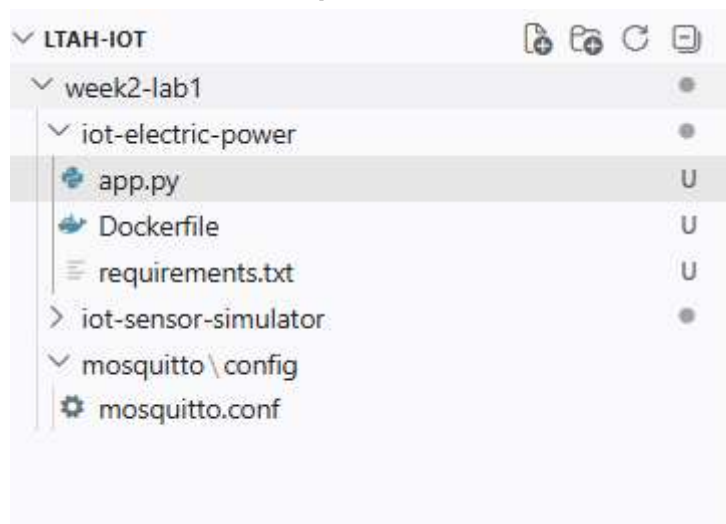
```
PS D:\LTAH-IoT> docker network inspect iot-net
[
  {
    "Name": "iot-net",
    "Id": "2e669b01ed2574d7aaa6839ed2f324cc5e6860523f10ff9ded20bda4690b2def",
    "Created": "2026-06-15T04:58:20.302453504Z",
    "Scope": "local",
    "Driver": "bridge",
    "EnableIPv4": true,
    "EnableIPv6": false,
    "IPAM": {
      "Driver": "default",
      "Options": {},
      "Config": [
        {
          "Subnet": "172.19.0.0/16",
          "Gateway": "172.19.0.1"
        }
      ]
    },
    "Internal": false,
    "Attachable": false,
    "Ingress": false,
    "ConfigFrom": {
      "Network": ""
    },
    "ConfigOnly": false,
    "Options": {
      "com.docker.network.enable_ipv4": "true",
      "com.docker.network.enable_ipv6": "false"
    },
    "Labels": {},
    "Containers": {
      "968f51119d356ad012aec8fd5c2732079698dc99f4733341d0da75e3023e679": {
        "Name": "sensor-01",
        "EndpointID": "34d4c968539903e5cadb4706fcd4f69021b8c2eb3ff552f3a5208535398c7c6",
        "MacAddress": "26:0e:99:39:0c:51",
        "IPv4Address": "172.19.0.3/16",
        "IPv6Address": ""
      },
      "9a5c67120897dc62196ab9fac8a9b262ffb717a87ae5dc6ba8760b44fa9c9ef": {
        "Name": "mos-mqtt-broker",
        "EndpointID": "12da308af6ce872f8254c0fd8516936e7bf7a02b35a9edf9d0db9efd21c508c0",
        "MacAddress": "16:44:71:af:ae:cd",
        "IPv4Address": "172.19.0.2/16",
        "IPv6Address": ""
      }
    }
  }
]
```

4. Dừng container:

```
PS D:\LTAH-IoT> docker stop sensor-01
sensor-01
PS D:\LTAH-IoT> █
```

XII. Phần 8: Tự xây dựng và Docker hóa ứng dụng IoT riêng

1. Cấu trúc thư mục và code:



- Source code app.py:

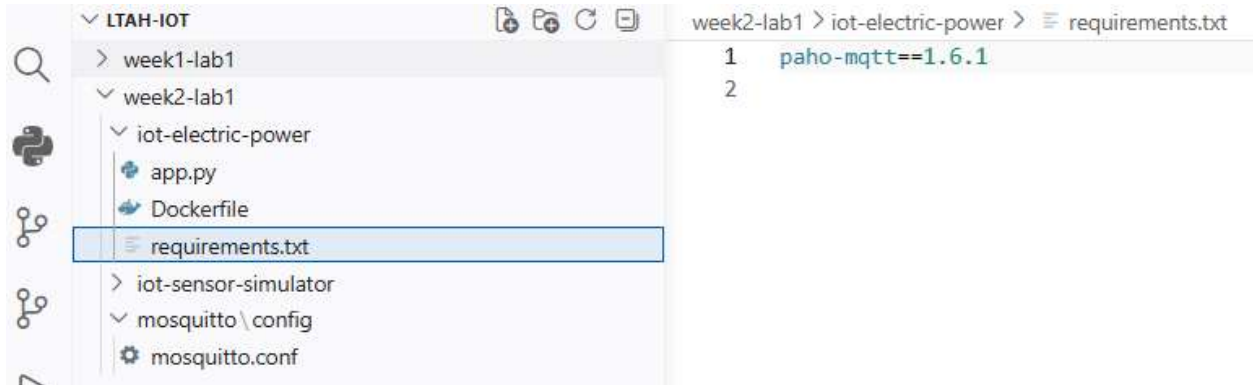
```
week2-lab1 > iot-electric-power > app.py > ...
1 import json
2 import os
3 import random
4 import time
5 from datetime import datetime, timezone
6 #datetime, timezone nằm trong module datetime của python
7
8 import paho.mqtt.client as mqtt
9
10 MQTT_BROKER = os.getenv('MQTT_BROKER', 'localhost') #địa chỉ của mqtt_broker: mặc định là localhost
11 MQTT_PORT = int(os.getenv('MQTT_PORT', '1883')) #os.getenv trả về string, dùng int(...) để ép về kiểu số nguyên
12 MQTT_TOPIC = os.getenv('MQTT_TOPIC', 'iot/power') #topic riêng cho dữ liệu công suất điện
13 DEVICE_ID = os.getenv('DEVICE_ID', 'power-meter-01') #mã định danh cho đồng hồ đo điện
14 PUBLISH_INTERVAL = float(os.getenv('PUBLISH_INTERVAL', '2'))
15
16 #điện áp danh định (V): lưới điện dân dụng Việt Nam là 220V
17 NOMINAL_VOLTAGE = float(os.getenv('NOMINAL_VOLTAGE', '220'))
18
19 #Trong python, sau khi khai báo hàm với dấu : ở cuối,
20 #tất cả các dòng thuộc thân hàm phải được thụt vào trong (thường là 4 dấu cách)
21 def create_power_data():
22     #tạo giá trị mô phỏng cho đồng hồ đo công suất điện
23     voltage = round(random.uniform(215.0, 225.0), 2) #điện áp (V), dao động quanh mức danh định 220V
24     current = round(random.uniform(0.5, 10.0), 2) #dòng điện (A)
25     power_factor = round(random.uniform(0.85, 1.0), 2) #hệ số công suất (cos phi), nằm trong khoảng 0..1
26
27     #công suất tiêu thụ thực tế P = U * I * cos(phi), đơn vị watt (W)
28     #làm tròn 2 chữ số cho gọn
29     power = round(voltage * current * power_factor, 2)
30
31     frequency = round(random.uniform(49.9, 50.1), 2) #tần số lưới điện (Hz), chuẩn VN là 50Hz
32
33     #trong dictionary (từ điển) phải dùng dấu , chứ không phải dấu ;
34     #dictionary là 1 kiểu dữ liệu trong Python để lưu trữ dữ liệu theo dạng cặp key: value
35     data = {
36         'device_id': DEVICE_ID,
37         'voltage': voltage, #điện áp (V)
38         'current': current, #dòng điện (A)
39         'power': power, #công suất tiêu thụ (W)
40         'power_factor': power_factor, #hệ số công suất
41         'frequency': frequency, #tần số (Hz)
42         'timestamp': datetime.now(timezone.utc).isoformat()
43     }
44     return data
45
46 def main(): #định nghĩa hàm main, là hàm điều chỉnh toàn bộ luồng chạy của chương trình (kết nối -> tạo dữ liệu -> gửi đi)
47     #tạo 1 mqtt client
48     client = mqtt.Client(client_id=DEVICE_ID)
49     #in thông báo ra màn hình nhờ f-string - cho phép chèn giá trị biến vào trong {...}
50     print(f'Connecting to mqtt broker at {MQTT_BROKER}:{MQTT_PORT}...')
51     #thực hiện kết nối tới broker, mở kết nối 60s, nếu trong 60s không có dữ liệu trao đổi, client sẽ ping để báo 'tôi vẫn sống',
52     #nếu broker không nhận được -> coi như client mất kết nối
53     client.connect(MQTT_BROKER, MQTT_PORT, keepalive=60)
54
55     #khởi động network loop chạy nền: hoàn tất bắt tay kết nối và giữ kết nối sống
56     #(xử lý ping/keepalive, gửi/nhận gói tin). Thiếu dòng này, publish sẽ báo lỗi rc=4 (NO_CONN)
57     client.loop_start()
58
59     print(f"Publishing data to topic: {MQTT_TOPIC}")
60
61     while True:
62         data = create_power_data()
63         payload = json.dumps(data)
64
65         result = client.publish(MQTT_TOPIC, payload)
66         if result.rc == mqtt.MQTT_ERR_SUCCESS:
67             print(f'Published: {payload}')
68         else:
69             print(f'Failed to publish message. Error code: {result.rc}')
70
71         time.sleep(PUBLISH_INTERVAL)
72
73 if __name__ == '__main__':
74     main()
75
```



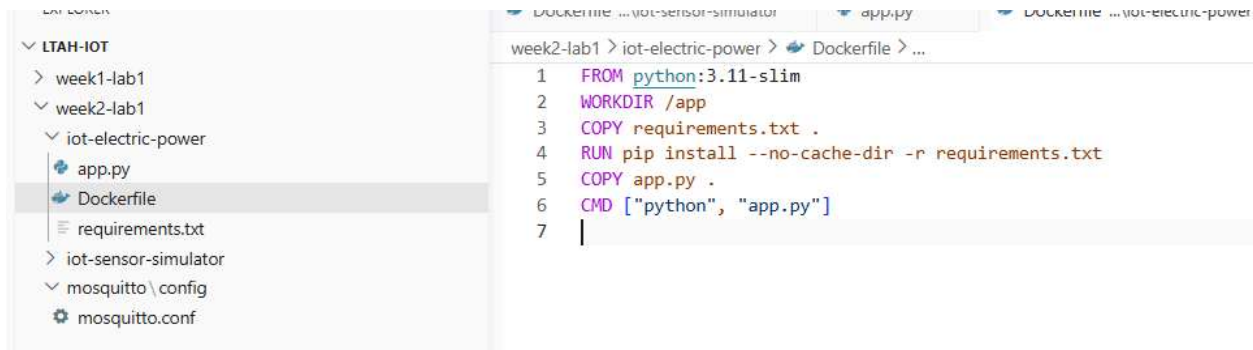
Snipping Tool

Screenshot copied to clipboard

- Requirements.txt: các thư viện cần cài đặt:



- Dockerfile:



2. Công dụng:

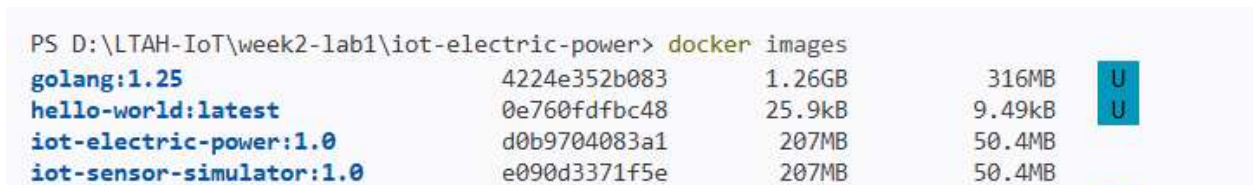
Theo dõi công suất điện

3. Kết quả chạy:

- Tạo image của iot-electric-power: `docker build -t iot-electric-power:1.0 .`



- Kiểm tra xem image đã được tạo chưa: `docker images` (check thấy tên -> đã được tạo)



- Chạy mos-mqtt-broker trong mạng iot-net: `docker run -it --rm --name mos-mqtt-broker --network iot-net -p 1883:1883 eclipse-mosquitto:2`

```
PS D:\LTAH-IoT\week2-lab1> docker run -it --rm --name mos-mqtt-broker --network iot-net -p 1883:1883 -v "$PWD/mosquitto/config:/mosquitto/config:ro" eclipse-mosquitto
:2
1781500470: Client auto-964E3793-42B1-71C9-4C95-E40530E81C81 [172.19.0.1:50372] disconnected: connection closed by client.
1781500473: Client sensor-02 [172.19.0.5:48119] disconnected: connection closed by client.
1781500478: New connection from 172.19.0.5:32917 on port 1883.
1781500478: New client connected from 172.19.0.5:32917 as sensor-02 (p4, c1, k60).
1781500571: New connection from 172.19.0.3:36645 on port 1883.
1781500571: New client connected from 172.19.0.3:36645 as sensor-01 (p4, c1, k60).
1781500700: Client sensor-01 [172.19.0.3:36645] disconnected: connection closed by client.
1781504804: Client sensor-02 [172.19.0.5:32917] disconnected: connection closed by client.
1781504806: Client sensor-03 [172.19.0.4:51319] disconnected: connection closed by client.
```

- Chạy sensor power-01 (1 container của iot-electric-power) trong mạng iot-net, publish các thông tin liên quan đến dụng cụ điện 01 (topic iot/power): `docker run -it --rm --name power-01 --network iot-net -e MQTT_BROKER=mos-mqtt-broker -e MQTT_PORT=1883 -e MQTT_TOPIC=iot/power -e DEVICE_ID=power-01 -e PUBLISH_INTERVAL=2 iot-electric-power:1.0`

```
PS D:\LTAH-IoT\week2-lab1\iot-electric-power> docker run -it --rm --name power-01 --network iot-net -e MQTT_BROKER=mos-mqtt-broker -e MQTT_PORT=1883 -e MQTT_TOPIC=iot/power -e DEVICE_ID=power-01 -e PUBLISH_INTERVAL=2 iot-electric-power:1.0
>>
Connecting to mqtt broker at mos-mqtt-broker:1883...
Publishing data to topic: iot/power
Published: {"device_id": "power-01", "voltage": 215.16, "current": 1.19, "power": 253.48, "power_factor": 0.99, "frequency": 49.94, "timestamp": "2026-06-15T06:44:59.903629+00:00"}
Published: {"device_id": "power-01", "voltage": 219.32, "current": 3.37, "power": 672.59, "power_factor": 0.91, "frequency": 50.01, "timestamp": "2026-06-15T06:45:01.904316+00:00"}
Published: {"device_id": "power-01", "voltage": 217.33, "current": 8.48, "power": 1566.51, "power_factor": 0.85, "frequency": 49.95, "timestamp": "2026-06-15T06:45:03.904941+00:00"}
```

- Chạy sensor power-02 (1 container của iot-electric-power) trong mạng iot-net, publish các thông tin liên quan đến dụng cụ điện 02 (topic iot/power): `docker run -it --rm --name power-02 --network iot-net -e MQTT_BROKER=mos-mqtt-broker -e MQTT_PORT=1883 -e MQTT_TOPIC=iot/power -e DEVICE_ID=power-02 -e PUBLISH_INTERVAL=2 iot-electric-power:1.0`

```
PS D:\LTAH-IoT\week2-lab1\iot-electric-power> docker run -it --rm --name power-02 --network iot-net -e MQTT_BROKER=mos-mqtt-broker -e MQTT_PORT=1883 -e MQTT_TOPIC=iot/power -e DEVICE_ID=power-02 -e PUBLISH_INTERVAL=2 iot-electric-power:1.0
Connecting to mqtt broker at mos-mqtt-broker:1883...
Publishing data to topic: iot/power
Published: {"device_id": "power-02", "voltage": 222.33, "current": 0.67, "power": 131.09, "power_factor": 0.88, "frequency": 49.98, "timestamp": "2026-06-15T06:45:30.127117+00:00"}
Published: {"device_id": "power-02", "voltage": 221.37, "current": 8.65, "power": 1838.26, "power_factor": 0.96, "frequency": 50.02, "timestamp": "2026-06-15T06:45:32.127682+00:00"}
```

- Chạy sensor power-03 (1 container của iot-electric-power) trong mạng iot-net, publish các thông tin liên quan đến dụng cụ điện 03 (topic iot/power): `docker run -it --rm --name power-03 --network iot-net -e MQTT_BROKER=mos-mqtt-broker -e MQTT_PORT=1883 -e MQTT_TOPIC=iot/power -e DEVICE_ID=power-03 -e PUBLISH_INTERVAL=2 iot-electric-power:1.0`

```
PS D:\LTAH-IoT> docker run -it --rm --name power-03 --network iot-net -e MQTT_BROKER=mos-mqtt-broker -e MQTT_PORT=1883 -e MQTT_TOPIC=iot/power -e DEVICE_ID=power-03 -e PUBLISH_INTERVAL=2 iot-electric-power:1.0
Connecting to mqtt broker at mos-mqtt-broker:1883...
Publishing data to topic: iot/power
Published: {"device_id": "power-03", "voltage": 218.66, "current": 4.4, "power": 962.1, "power_factor": 1.0, "frequency": 50.05, "timestamp": "2026-06-15T06:45:50.258935+00:00"}
Published: {"device_id": "power-03", "voltage": 216.58, "current": 8.76, "power": 1631.63, "power_factor": 0.86, "frequency": 49.92, "timestamp": "2026-06-15T06:45:52.264818+00:00"}
Published: {"device_id": "power-03", "voltage": 220.62, "current": 1.83, "power": 375.47, "power_factor": 0.93, "frequency": 49.93, "timestamp": "2026-06-15T06:45:54.267508+00:00"}
```

- Tạo 1 subscriber lắng nghe thông tin từ topic iot/power: subscriber nhận thông tin của cả 3 sensor: `mosquitto_sub -h localhost -p 1883 -t 'iot/power'`


```

Use 'mosquitto_sub --help' to see usage.
thuily05@Thuily08:/mnt/c/WINDOWS/system32$ mosquitto_sub -h localhost -p 1883 -t 'iot/power'
{"device_id": "power-01", "voltage": 215.58, "current": 4.84, "power": 949.5, "power_factor": 0.91, "frequency": 49.94,
"timestamp": "2026-06-15T06:48:11.953662+00:00"}
{"device_id": "power-02", "voltage": 215.38, "current": 7.74, "power": 1500.34, "power_factor": 0.9, "frequency": 49.92,
"timestamp": "2026-06-15T06:48:12.169792+00:00"}
{"device_id": "power-03", "voltage": 219.06, "current": 8.05, "power": 1622.36, "power_factor": 0.92, "frequency": 49.99,
"timestamp": "2026-06-15T06:48:12.304556+00:00"}
{"device_id": "power-01", "voltage": 217.31, "current": 4.9, "power": 1054.17, "power_factor": 0.99, "frequency": 50.01,
"timestamp": "2026-06-15T06:48:13.952808+00:00"}
{"device_id": "power-02", "voltage": 218.33, "current": 3.38, "power": 723.2, "power_factor": 0.98, "frequency": 50.01,
"timestamp": "2026-06-15T06:48:14.168734+00:00"}
{"device_id": "power-03", "voltage": 220.08, "current": 6.6, "power": 1307.28, "power_factor": 0.9, "frequency": 50.04,
"timestamp": "2026-06-15T06:48:14.303575+00:00"}
{"device_id": "power-01", "voltage": 215.9, "current": 5.03, "power": 933.94, "power_factor": 0.86, "frequency": 50.04,
"timestamp": "2026-06-15T06:48:15.953388+00:00"}
{"device_id": "power-02", "voltage": 217.95, "current": 1.94, "power": 384.77, "power_factor": 0.91, "frequency": 49.97,
"timestamp": "2026-06-15T06:48:16.169249+00:00"}
{"device_id": "power-03", "voltage": 223.62, "current": 6.04, "power": 1310.14, "power_factor": 0.97, "frequency": 49.99,
"timestamp": "2026-06-15T06:48:16.304180+00:00"}

```

XIII. Câu hỏi báo cáo

1. MQTT broker có vai trò gì trong hệ thống IoT?

MQTT broker là **máy chủ trung gian** đứng giữa các thiết bị. Mọi thiết bị không gửi dữ liệu trực tiếp cho nhau mà gửi qua broker. Broker nhận message từ bên gửi (publisher) theo từng ***topic***, rồi chuyển tiếp đến tất cả các bên đã đăng ký nhận (subscriber) của topic đó. Nhờ vậy thiết bị gửi và thiết bị nhận **không cần biết về nhau** (giảm phụ thuộc), dễ mở rộng số lượng thiết bị, và quản lý kết nối tập trung tại một chỗ.

2. Sự khác nhau giữa MQTT publisher và MQTT subscriber là gì?

- Publisher (bên xuất bản): gửi/đẩy dữ liệu lên broker theo một topic. Trong lab này, `app.py` đóng vai trò publisher — nó tạo dữ liệu cảm biến và `publish` lên topic `iot/sensors`.
- Subscriber (bên đăng ký): đăng ký một topic và nhận mọi message được publish lên topic đó.
- Một thiết bị có thể vừa là publisher vừa là subscriber. Điểm khác nhau cốt lõi: publisher - gửi đi, subscriber - nhận về.

3. Vì sao cần dùng Docker container để đóng gói ứng dụng IoT?

- Đồng nhất môi trường: container đóng gói sẵn code + thư viện + cấu hình, nên chạy ở máy nào cũng giống nhau, tránh lỗi "máy tôi chạy được mà máy bạn không".
- Cô lập: mỗi service (broker, sensor) chạy trong container riêng, không xung đột thư viện/hệ điều hành.
- Dễ triển khai & mở rộng: chỉ cần một lệnh để bật/tắt/nhân bản nhiều sensor.
- Gọn nhẹ & nhanh hơn máy ảo vì chia sẻ chung nhân hệ điều hành.

4. Trong lệnh chạy broker, tham số **-p 1883:1883** có ý nghĩa gì?

- p là port mapping (ánh xạ cổng) giữa máy thật và container, theo dạng **-p <cổng máy thật>:<cổng trong container>**. 1883:1883 nghĩa là chuyển tiếp cổng 1883 của máy host vào cổng 1883 bên trong container (1883 là cổng mặc định của MQTT). Nhờ đó ứng dụng bên ngoài container có thể kết nối tới broker qua localhost:1883.

5. Trong lệnh chạy sensor container, vì sao MQTT_BROKER được đặt là **mqtt-broker** thay vì localhost?

Vì khi chạy trong Docker, mỗi container có mạng riêng. Với một container, localhost chỉ về chính nó, không phải broker. Khi các container nằm chung một Docker network, Docker có DNS nội bộ cho phép gọi nhau bằng tên service/tên container. mqtt-broker chính là tên của

container broker, nên sensor container phải dùng tên đó để Docker phân giải ra đúng địa chỉ của broker.

6. Docker network `iot-net` có vai trò gì?

Nó tạo một mạng ảo riêng để các container (broker và sensor) cùng tham gia. Trong mạng này, các container có thể:

- Gọi nhau bằng tên thay vì IP (nhờ DNS nội bộ của Docker).
- Giao tiếp trực tiếp và cô lập với các container/mạng khác (an toàn hơn).

7. Nếu không dùng Docker network riêng, sensor container có thể kết nối đến broker bằng tên `mqtt-broker` không? Giải thích.

Không. Việc phân giải tên container thành địa chỉ chỉ hoạt động khi các container ở chung một user-defined network. Mạng bridge mặc định của Docker không có tính năng DNS tự động theo tên. Nếu không tạo network riêng, sensor sẽ không hiểu `mqtt-broker` là gì và sẽ báo lỗi không phân giải được tên → kết nối thất bại.

8. Vì sao chương trình Python nên cấu hình từ biến môi trường thay vì hard-code trong source code?

- Linh hoạt: đổi địa chỉ broker, port, topic, chu kỳ gửi... mà không phải sửa và build lại code.
- Tách cấu hình khỏi code: cùng một image chạy được ở nhiều môi trường (dev/test/prod) chỉ bằng cách đổi biến môi trường.
- An toàn: thông tin nhạy cảm (mật khẩu, địa chỉ) không bị ghi cứng trong source.
- Đúng tinh thần của Docker/12-factor app. Trong `app.py`, ví dụ `os.getenv('MQTT_BROKER', 'localhost')` đọc cấu hình từ môi trường, có giá trị mặc định khi không được đặt.

9. Nếu muốn thay đổi chu kỳ gửi dữ liệu từ 2 giây sang 10 giây mà không sửa code, cần làm gì?

- Đặt biến môi trường `PUBLISH_INTERVAL=10` khi chạy container (hoặc trong file `docker-compose`).
- Vì code đã đọc `PUBLISH_INTERVAL = float(os.getenv('PUBLISH_INTERVAL', '2'))`, nó sẽ tự lấy giá trị 10 mà không cần đụng đến source.

10. Trong hệ thống IoT thật, vì sao không nên để `allow_anonymous true` cho MQTT broker?

`allow_anonymous true` cho phép bất kỳ ai cũng kết nối được broker mà không cần đăng nhập. Trong môi trường thật, điều này nguy hiểm:

- Kẻ xấu có thể đọc trộm dữ liệu cảm biến (subscribe topic).
- Có thể gửi dữ liệu giả lên topic, gây sai lệch/điều khiển sai thiết bị.
- Có thể làm quá tải broker (tấn công).

Hệ thống thật nên bật xác thực (username/password hoặc chứng chỉ TLS) và phân quyền theo topic. `allow_anonymous true` chỉ nên dùng khi học/thử nghiệm trong mạng nội bộ.

11. Ứng dụng IoT riêng của bạn publish dữ liệu lên topic nào? Vì sao bạn chọn topic đó?

Ứng dụng publish lên topic `iot/power` (cấu hình qua biến môi trường `MQTT_TOPIC`).

Lý do chọn topic này:

- Cấu trúc phân cấp rõ ràng: nhóm `iot` cho toàn hệ thống, nhánh `power` dành riêng cho dữ liệu công suất điện — tách biệt hẳn với `iot/sensors` (nhiệt độ/độ ẩm).

- Dễ phân loại & lọc: subscriber muốn theo dõi riêng dữ liệu điện chỉ cần đăng ký `iot/power`, không bị lẫn với dữ liệu cảm biến khác.
- Dễ mở rộng: sau này có thể chia nhỏ thành `iot/power/voltage`, `iot/power/current...` hoặc đăng ký gộp bằng wildcard `iot/#` để nhận tất cả.

12. Ứng dụng IoT riêng của bạn gửi những trường dữ liệu nào trong JSON message?

JSON message gồm 7 trường:

Trường	Ý nghĩa	Đơn vị
<code>device_id</code>	Mã định danh đồng hồ đo điện (vd <code>power-meter-01</code>)	—
<code>voltage</code>	Điện áp	V (vôn)
<code>current</code>	Dòng điện	A (ampe)
<code>power</code>	Công suất tiêu thụ thực tế, tính theo $P = U \times I \times \cos(\varphi)$	W (watt)
<code>power_factor</code>	Hệ số công suất $\cos(\varphi)$, nằm trong khoảng 0–1	—
<code>frequency</code>	Tần số lưới điện (chuẩn VN là 50Hz)	Hz
<code>timestamp</code>	Thời điểm tạo dữ liệu, theo chuẩn ISO 8601 (UTC)	—

13. Vai trò của thư viện psutil là gì trong ứng dụng gửi thông tin hệ thống?

`psutil` (Python System and Process Utilities) là thư viện dùng để lấy thông tin tài nguyên hệ thống như: phần trăm sử dụng CPU, dung lượng RAM, ổ đĩa, mạng, tiến trình đang chạy... Trong một ứng dụng gửi thông tin hệ thống, `psutil` đảm nhận việc thu thập các chỉ số phần cứng/hệ điều hành, sau đó dữ liệu này được đóng gói (JSON) và publish lên broker.

Lưu ý: ứng dụng sensor trong lab này dùng `paho-mqtt` để giao tiếp MQTT và random để mô phỏng dữ liệu, chưa dùng `psutil`. `psutil` chỉ cần khi muốn gửi số liệu thật của máy.

14. Khi chạy ứng dụng IoT riêng trong container, vì sao vẫn cần đặt `MQTT_BROKER=mqtt-broker`?

Vì ứng dụng chạy trong container của nó, `localhost` (giá trị mặc định trong code) sẽ trở về chính container đó chứ không phải broker. Để kết nối được tới container broker, ta phải trở đúng tới tên service/container của broker trong cùng Docker network — đó là `mqtt-broker`. Vì vậy phải đặt `MQTT_BROKER=mqtt-broker` để ghi đè giá trị mặc định `localhost`, giúp Docker DNS phân giải đúng địa chỉ broker.

XIV. Dọn dẹp môi trường

- Xóa các sensor và mqtt broker: `docker stop sensor-01 sensor-02 sensor-03 power-01 power-02 power-03 mos-mqtt-broker`

```
PS D:\LTAH-IoT> docker stop sensor-01 sensor-02 sensor-03 power-01 power-02 power-03 mos-mqtt-broker
mos-mqtt-broker
Error response from daemon: No such container: sensor-01
Error response from daemon: No such container: sensor-02
Error response from daemon: No such container: sensor-03
Error response from daemon: No such container: power-01
Error response from daemon: No such container: power-02
Error response from daemon: No such container: power-03
```

- Xóa mạng `iot-net`: `docker network rm iot-net`

```
PS D:\LTAH-IoT> docker network rm iot-net
iot-net
PS D:\LTAH-IoT> █
```

- Xóa images: `docker rmi iot-sensor-simulator:1.0 iot-electric-power:1.0`

```
PS D:\LTAH-IoT> docker rmi iot-sensor-simulator:1.0 iot-electric-power:1.0
Untagged: iot-sensor-simulator:1.0
Deleted: sha256:e090d3371f5ec49512b100e0b48db57e3408f853da57d38a9d2c0549f284350e
Untagged: iot-electric-power:1.0
Deleted: sha256:d0b9704083a1bad4a91c343cd2fbfca37978b2cdec259e5ab5e613e735716793
PS D:\LTAH-IoT>
```

XV. Kết luận

Trong bài thực hành này, sinh viên đã xây dựng một hệ thống IoT tối thiểu với MQTT broker và các cảm biến giả lập. Thông qua Docker, các thành phần của hệ thống được đóng gói và chạy độc lập hoàn toàn. Đây là nền tảng quan trọng để chuyển sang Buổi 2, nơi toàn bộ hệ thống IoT sẽ được tổ chức dưới dạng multi-container application bằng Docker Compose, kết hợp với InfluxDB & Grafana để lưu trữ và trực quan hóa dữ liệu cảm biến.